1

Создание хранимых процедур

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Рассматриваемые вопросы

- Определение и создание процедуры
- Создание процедур с параметрами
- Различия между формальными и действительными параметрами
- Использование параметров различного типа
- Вызов процедуры
- Обработка исключений в процедуре
- Удаление процедуры

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Цель урока

На этом уроке вы научитесь создавать, выполнять и удалять процедуры с параметрами и без параметров. Процедуры составляют основу модульного программирования на PL/SQL. Для того, чтобы они были более гибкими в них следует передавать подсчитываемые значения или данные в качестве входных параметров. С помощью параметров типа `OUT` можно передавать результаты в вызывающую программу или среду.

Для создания надежных программ необходимо всегда определять обработку возникающих ошибок, используя возможности обработки исключений, предоставляемые PL/SQL.

Что такое процедура?

- **Процедура – это вид подпрограммы, выполняющей действия.**
- **Процедура может храниться в базе данных как объект схемы.**
- **Использование процедур позволяет повторно выполнять и сопровождать код.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Определение процедуры

Процедура (*procedure*) – это именованный блок PL/SQL, который может принимать параметры (иногда их называют аргументами). Обычно процедура используется для выполнения какого-либо действия. Процедура имеет заголовок, декларативную секцию, исполняемую секцию и необязательную секцию обработки исключений. Процедура вызывается по имени из исполняемой секции другого блока PL/SQL.

Процедура может быть скомпилирована и записана в базу данных как объект схемы. При использовании процедур в Oracle Forms и Reports они компилируются в среде этих инструментальных средств.

Процедуры обеспечивает повторную используемость и упрощают сопровождение кода. Проверенная процедура может использоваться любым количеством приложений. При изменении требований необходимо изменить только процедуру.

Создание процедур: синтаксис

- Укажите после `CREATE PROCEDURE` имя, необязательные параметры и ключевое слово `IS` или `AS`.
- Добавьте необязательную опцию `OR REPLACE`, чтобы перезаписать существующую процедуру.
- Напишите код блока PL/SQL, укажите локальные переменные, ключевые слова `BEGIN`, `END` (или `END` *имя_процедуры*).

```
CREATE [OR REPLACE] PROCEDURE имя_процедуры
 [(параметр1 [режим] тип_данных1,
   параметр2 [режим] тип_данных2, ...)]
IS|AS
  [объявления_локальных_переменных; …]
BEGIN
  -- действия;
END [имя_процедуры];
```

Блок PL/SQL

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Создание процедур: синтаксис

Для создания новой процедуры используется команда `CREATE PROCEDURE`, которая может содержать список объявляемых параметров и действия, которые должен выполнять стандартный блок PL/SQL. По этой команде создается автономная (stand-alone) процедура, хранимая в базе данных Oracle.

- *Блок PL/SQL* (*PL/SQL block*) начинается с ключевого слова `BEGIN`, за которым необязательно следует объявление локальных переменных, а заканчивается `END` или `END` *имя_процедуры*.
- *Опция* `REPLACE` (`REPLACE` *option*) означает, что, если процедура с таким именем уже существует, она будет удалена, а вместо нее будет использоваться новый вариант, создаваемый данной командой.

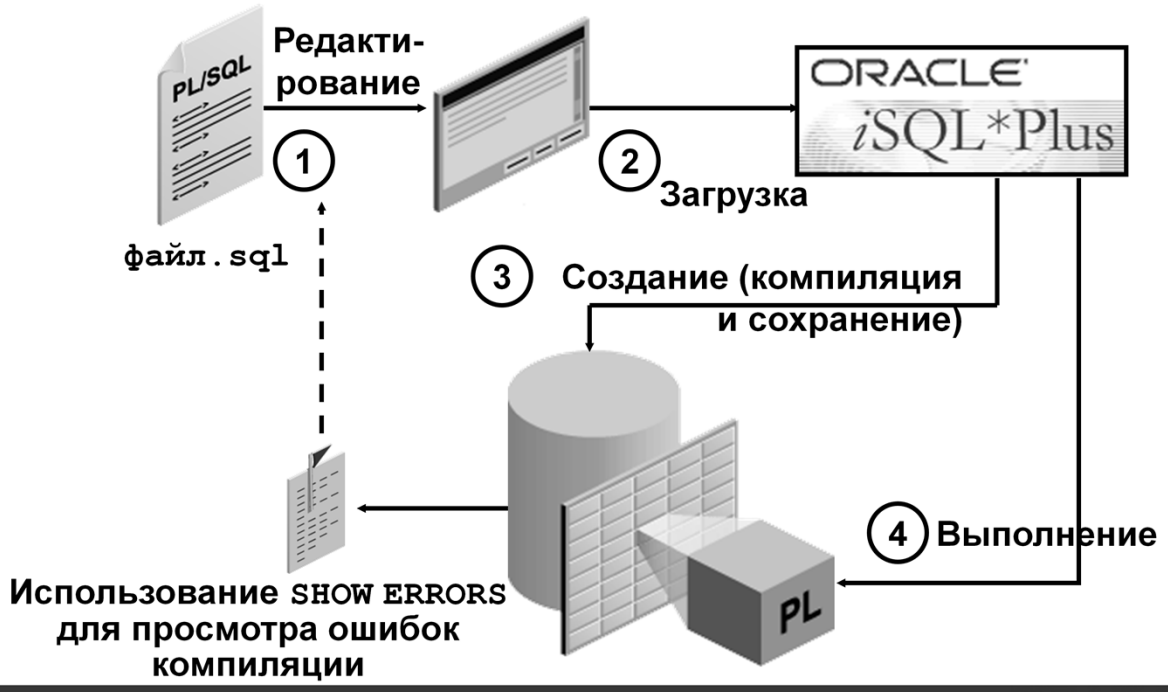
Остальные синтаксические элементы

- *параметр1* представляет собой имя параметра.
- *режим или* `тип_параметра` (*mode*) определяет то, как используется параметр: `IN` (по умолчаниют), `OUT` или `IN OUT`.
- *тип_данных1* задается для параметра без указания какой-либо точности.

Примечание. Параметры можно рассматривать в качестве локальных переменных. В PL/SQL- блоке хранимой процедуры нельзя ссылаться на хост-переменные или связанные переменные.

Использование опции `OR REPLACE` не требует никаких изменений в безопасности, если вы владелец объекта или имеете привилегию `CREATE [ANY] PROCEDURE`.

Разработка процедур



Copyright © 2004, Oracle. All rights reserved.

Разработка процедур

В ходе разработки хранимой процедуры следует выполнить следующие шаги:

1. Напишите код для создания процедуры в редакторе или текстовом процессоре, а затем сохраните его в скрипт-файле SQL (обычно имеющем расширение `.sql`).
2. Загрузите код в одно из средств разработки, например, SQL*Plus или *i*SQL*Plus.
3. Создайте процедуру в базе данных. Команда `CREATE PROCEDURE` компилирует и сохраняет исходный код и полученный после компиляции *m-код* в базе данных. В случае ошибок компиляции *m-код* не сохраняется и необходимо скорректировать исходный код. Нельзя вызвать процедуру, имеющую ошибки компиляции. Для их просмотра используйте:
   - `SHOW ERRORS`, чтобы вывести сообщения об ошибках последней скомпилированной процедуры;
   - `SHOW ERRORS PROCEDURE имя_процедуры`, чтобы вывести сообщения о любой ранее скомпилированной процедуры.
4. После успешной компиляции вызовите процедуру, выполняющую необходимые действия. Используйте команду `EXECUTE` в *i*SQL*Plus или анонимный блок в среде, поддерживающей PL/SQL.

Примечание. При наличии ошибок компиляции можно, используя команду `CREATE OR REPLACE PROCEDURE`, переписать существующий код, ранее созданный по команде `CREATE PROCEDURE`. Другой способ: сначала удалить, а затем создать процедуру, используя команды `DROP` и `CREATE PROCEDURE`.

Что такое параметры?

Параметры:

- **Объявляются после имени подпрограммы в заголовке PL/SQL**
- **Передают данные из вызывающей программы или среды и возвращают их из подпрограммы**
- **Подобны локальным переменным, но используются в зависимости от их режима (типа):**
  - **Параметр `IN` (по умолчанию) предоставляет значение процессу, выполняющему подпрограмму.**
  - **Параметр `OUT` возвращает значение вызывающей программе или среде.**
  - **Параметр `IN OUT` предоставляет входное значение, которое может быть возвращено после изменения.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Что такое параметры?

Параметры (*parameters*) используются для передачи и возврата значений из вызывающей среды в процедуру (или подпрограмму). Параметры объявляются в заголовке подпрограммы после имени и перед декларативной секцией локальных переменных.

Параметры могут использоваться в трех различных режимах передачи: `IN`, `OUT` или `IN OUT`.

- Параметр `IN` передает значение константы из вызывающей среды в процедуру.
- Параметр `OUT` передает значение из процедуры в вызывающую среду.
- Параметр `IN OUT` передает значение из вызывающей среды в процедуру и возможно возвращает другое значение обратно из процедуры в вызывающую среду.

Параметры можно считать специальным видом локальных переменных. Их входные значения инициализируются в вызывающей среде при вызове подпрограммы. Выходные значения параметров возвращаются при передаче управления назад в вызывающую среду.

Формальные и фактические параметры

- **Формальные параметры: локальные переменные, объявленные в списке параметров спецификации подпрограммы**

  Пример:

```
CREATE PROCEDURE raise_sal(id NUMBER,sal NUMBER) IS
BEGIN ...
END raise_sal;
```

- **Фактические параметры: литеральные величины, переменные или выражения, используемые в списке параметров вызова подпрограммы**

  Пример:

```
emp_id := 100;
raise_sal(emp_id, 2000)
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Формальные и фактические параметры

Формальные параметры (*formal parameters*) – это локальные переменные, объявленные в списке параметров спецификации подпрограммы. Например, в процедуре `raise_sal` переменные `id` и `sal` – идентификаторы формальных параметров.

Фактические параметры (*actual parameters*) могут быть литеральными значениями, переменными или выражения, приведенными в списке параметров вызова подпрограммы. Во втором примере в вызове процедуры `raise_sal` переменная `emp_id` предоставляет фактическое значение для параметра `id`,а литерал `2000` – для `sal`. Фактические параметры:

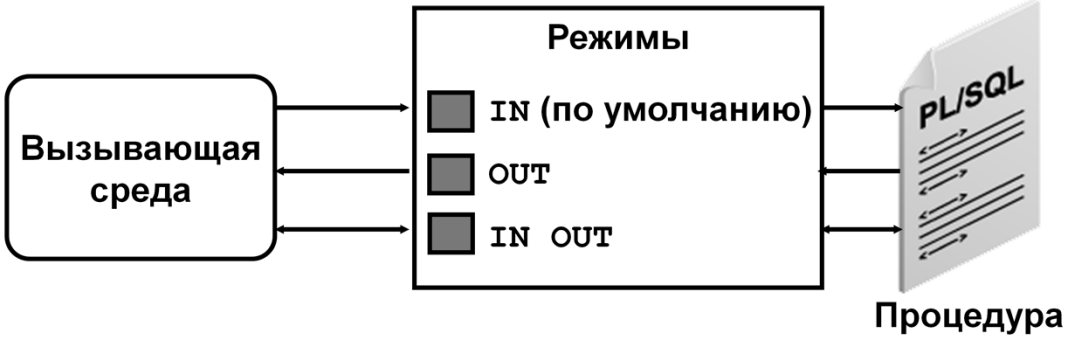
- связываются с формальными параметрами во время вызова подпрограммы;
- могут представлять собой выражения, например: `raise_sal(emp_id, raise+100);`

Формальные и фактические параметры должны иметь совместимые типы данных. Если это необходимо, PL/SQL преобразует тип данных фактического параметра перед его присваиванием формальному параметру.

Режимы процедурных параметров

- **Режимы указываются при объявлении формальных параметров после имени параметра и перед типом его данных.**
- **Если режим параметра не указан, значение по умолчанию – IN.**

```
CREATE PROCEDURE процедура(параметр[режим] тип_данных
...
```



Copyright © 2004, Oracle. All rights reserved.

Режимы процедурных параметров

Когда вы создаете процедуру, формальный параметр определяет имя переменной, значение которой используется в исполняемой секции блока PL/SQL. Фактический параметр используется в вызове процедуры и с его помощью передаются входные значения и принимаются выходные результаты.

IN – это режим передачи параметра по умолчанию. Когда явно не указывается режим (тип параметра), его значение – IN. Другие режимы параметров должны быть явно указаны (OUT или IN OUT) в определении каждого из таких параметров.

Тип_данных параметра не содержит спецификации размера. *Тип_данных* можно указать:

- явно;
- используя описание с атрибутом %TYPE;
- используя описание с атрибутом %ROWTYPE.

Примечание: через запятую можно объявить несколько параметров.

Пример использование параметров в режиме IN

```
CREATE OR REPLACE PROCEDURE raise_salary
  (id      IN employees.employee_id%TYPE,
   percent IN NUMBER)
IS
BEGIN
  UPDATE employees
  SET    salary = salary * (1 + percent/100)
  WHERE  employee_id = id;
END raise_salary;
/
```

```
EXECUTE raise_salary(176,10)
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Пример использование параметров в режиме IN

Пример на слайде показывает процедуру с двумя параметрами в режиме IN. В результате выполнения первой команды в *i*SQL*Plus в базе данных создается процедура raise_salary. Вторая команда вызывает процедуру raise_salary и передает ей в качестве первого параметра номер служащего (176), а в качестве второго параметра – процент повышения его оклада (10%).

Для *вызова процедуры* (*invoke a procedure*) в *i*SQL*Plus используйте команду EXECUTE:

```
EXECUTE raise_salary (176, 10)
```

Чтобы вызвать процедуру из другой процедуры, используйте прямой вызов в выполняемой секции вызывающего блока. В месте вызова процедуры введите имя этой процедуры и ее фактические параметры. Пример:

```
...
BEGIN
  raise_salary (176, 10);
END;
```

Примечание. Параметры в режиме IN передаются из вызывающей среды в процедуру в виде значений, доступных только на чтение. Попытка изменить значение параметра типа IN приводит к ошибке компиляции.

Пример использование параметров в режиме OUT

```
CREATE OR REPLACE PROCEDURE query_emp
(id     IN  employees.employee_id%TYPE,
 name   OUT employees.last_name%TYPE,
 salary OUT employees.salary%TYPE) IS
BEGIN
  SELECT   last_name, salary INTO name, salary
   FROM    employees
   WHERE   employee_id = id;
END query_emp;
```

```
DECLARE
  emp_name employees.last_name%TYPE;
  emp_sal  employees.salary%TYPE;
BEGIN
  query_emp(171, emp_name, emp_sal); ...
END;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Пример использование параметров в режиме OUT

В этом примере создается процедура с параметрами, используемыми в режиме OUT для получения сведений о сотруднике. Процедура принимает номер служащего (171) в качестве значения параметра ID. Фамилия и оклад для соответствующего номера служащего выбираются и передаются в вызывающую среду с помощью двух параметров типа OUT. Процедура query_emp имеет три формальных параметра. Два из них – это параметры типа OUT, с помощью которых значения возвращаются в переменные emp_name and emp_salary вызывающей среды (как это показано в коде, приведенном в нижней части слайда).

В результате выполнения в вызывающем блоке переменные имеют значения:

- emp_name содержит значение Smith;
- emp_salary содержит значение 7600.

Примечание: необходимо, чтобы переменные, задаваемые в качестве фактических параметров и используемые для получения значений из параметров типа OUT, имели размер, достаточный для хранения возвращаемых значений.

При попытке использования или чтения параметров типа OUT внутри процедуры, в которой они объявлены, выдается ошибка компиляции. Параметрам типа OUT в теле процедуры, в которой они объявлены, могут быть только присвоены значения.

Просмотр параметров типа OUT в *i*SQL*Plus

- **Использование переменных PL/SQL, значения которых выводятся с помощью процедуры DBMS_OUTPUT.PUT_LINE.**

```
SET SERVEROUTPUT ON
DECLARE
  emp_name employees.last_name%TYPE;
  emp_sal  employees.salary%TYPE;
BEGIN
  query_emp(171, emp_name, emp_sal);
  DBMS_OUTPUT.PUT_LINE('Name: ' || emp_name);
  DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_sal);
END;
```

- **Использование хост-переменных *i*SQL*Plus в процедуре QUERY_EMP и вывод их значений.**

```
VARIABLE name VARCHAR2(25)
VARIABLE sal  NUMBER
EXECUTE query_emp(171, :name, :sal)
PRINT name sal
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Просмотр параметров типа OUT в среде *i*SQL*Plus

Примеры демонстрируют два способа просмотра значений, возвращаемых параметрами типа OUT.

- В первом способе в анонимном блоке используются переменные PL/SQL для получения значений, возвращаемых параметрами типа OUT. Для вывода значений переменных PL/SQL используется процедура DBMS_OUPUT.PUT_LINE. В сеансе до анонимного блока должна быть выполнена команда SET SERVEROUPUT ON.
- Второй способ основывается на использовании переменных *i*SQL*Plus, создаваемых по команде VARIABLE. Такие внешние по отношению к блоку PL/SQL переменные называют *хост-переменными* (*host variables*) или *связанными переменными* (*bind variables*). Вы можете ссылаться на хост-переменные в блоке PL/SQL. Перед ссылкой на хост-переменную ставится двоеточие (:). Для вывода значений связанных переменных используется команда *i*SQL*Plus PRINT, в которой имена переменных *i*SQL*Plus указываются без двоеточия (поскольку PRINT не является командой PL/SQL).

Шаги применения хост-переменных при вызове процедуры с параметрами типа OUT:

1. С помощью редактора создайте скрипт-файл *i*SQL*Plus.
2. Добавьте в скрипт команды для создания переменных, выполнения процедуры и вывода переменных.
3. Загрузите и выполните скрипт-файл в *i*SQL*Plus.

Примечание: дополнительные сведения о команде VARIABLE см. в документе *i*SQL*Plus Command Reference.

Вызов кода PL/SQL с использованием хост-переменных

Хост-переменная (также называемая связанной или глобальной переменной):

- **Объявляется и существует вне подпрограммы PL/SQL. Хост-переменная может быть создана:**
  - **в *i*SQL*Plus по команде `VARIABLE`**
  - **в Oracle Forms или как переменная UI (user interface, пользовательский интерфейс)**
  - **как Java-переменная.**
- **В коде PL/SQL перед именем хост-переменной ставится двоеточие (`:`).**
- **На хост-переменную можно ссылаться в анонимном блоке, но не в хранимой процедуре.**
- **С помощью хост-переменной значение передается и возвращается из блока PL/SQL.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Вызов кода PL/SQL с использованием хост-переменных

Код PL/SQL, хранимый в базе данных, можно вызвать из разных сред, например:

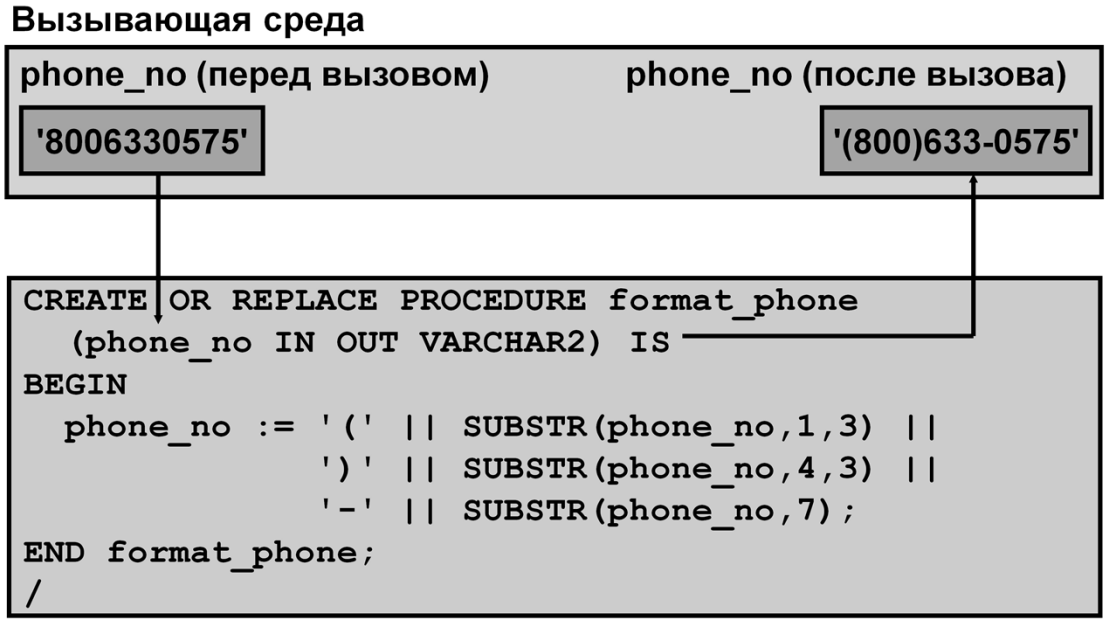
- SQL*Plus или *i*SQL*Plus;
- Oracle Forms или Oracle Reports;
- Приложений на Java или C.

Каждая их таких сред предоставляет возможности объявления переменных для хранения данных в памяти. Эти переменные объявляются и содержатся вне хранимого кода PL/SQL. Каждая среда предоставляет способ передачи переменной в код PL/SQL и возврата из него измененных значений. Обычно блоки PL/SQL или подпрограммы можно вызвать из большинства хост-языков. Машина PL/SQL использует метод, называемый *связыванием* (*binding*), чтобы сопоставить переменные или параметры, объявленные в подпрограммах PL/SQL, с значениями, предоставляемыми из внешних мест расположения.

В отличие от Java в PL/SQL хост-переменные распознаются с помощью двоеточия перед именами внешних переменных.

Нельзя сохранить код PL/SQL с хост-переменными, так как компилятор не может разрешить ссылки на хост-переменные. Процесс связывания происходит во время выполнения.

Пример использование параметров в режиме IN OUT



Copyright © 2004, Oracle. All rights reserved.

Пример использование параметров в режиме IN OUT

С помощью параметра типа IN OUT в процедуру можно передать значение, которое затем в этой процедуре может быть изменено. Фактический параметр, предоставленный из вызывающей среды, может содержать при возврате:

- первоначальное неизмененное значение;
- новое значение, установленное внутри процедуры.

Примечание: параметр типа IN OUT выступает в качестве инициализированной переменной.

В примере на слайде создается процедура с параметром типа IN OUT для приема 10-символьной строки, содержащей цифры номера телефона. Процедура возвращает номер телефона, отформатированный с помощью круглых скобок вокруг первых трех символов и черточки после шестой цифры, например, строка с номером телефона 8006330575 возвращается в виде: (800) 633-0575.

В следующем коде используется хост-переменная *i*SQL*Plus phone_no, чтобы предоставить входное значение процедуре FORMAT_PHONE. После выполнения процедуры в переменной phone_no возвращается измененная строка.

```
VARIABLE phone_no VARCHAR2(15)
EXECUTE :phone_no := '8006330575'
PRINT phone_no
EXECUTE format_phone (:phone_no)
PRINT phone_no
```

Синтаксис передачи параметров

- **Позиционно:**
  - **Фактические параметры перечисляются в том же порядке, что и формальные параметры.**
- **По имени параметра:**
  - **Фактические параметры перечисляются в произвольном порядке; связывание каждого именованного формального параметра с соответствующим фактическим параметром производится с помощью оператора =>.**
- **Комбинированно:**
  - **некоторые из фактических параметров перечисляются как позиционные и некоторые вместе с их именами.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Синтаксис передачи параметров

Для каждой процедуры, имеющей несколько параметров, можно использовать несколько методов указания значений этих параметров. К таким методам относятся следующие:

- *Позиционный*; значения фактических параметров указываются в том порядке, в котором были объявлены формальные параметры.
- *По имени параметра*; фактические значения указываются в произвольном порядке, но каждый фактический параметр связывается с именем формального параметра с помощью *оператора связывания* (*association operator*). Этот оператор в PL/SQL представляется как знак равенства, за которым без пробела следует знак больше (=>) .
- *Комбинированный*; первые значения указываются позиционно, а остальные – по имени параметра с помощью специального синтаксиса для метода задания параметров по имени.

Несколько примеров использования первых двух методов показано на следующей странице.

Примеры передачи параметров

```
CREATE OR REPLACE PROCEDURE add_dept(
  name IN departments.department_name%TYPE,
  loc  IN departments.location_id%TYPE) IS
BEGIN
  INSERT INTO departments(department_id,
            department_name, location_id)
  VALUES (departments_seq.NEXTVAL, name, loc);
END add_dept;
/
```

- **Позиционная форма записи передаваемых параметров**

```
EXECUTE add_dept ('TRAINING', 2500)
```

- **Использование имен передаваемых параметров**

```
EXECUTE add_dept (loc=>2400, name=>'EDUCATION')
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Примеры передачи параметров

В примере в процедуре `add_dept` объявлены два параметра типа `IN`: `name` и `loc`. Их значения используются в команде `INSERT` для задания наименования отдела (`department_name`) и кода его расположения (`location_id`).

Позиционная форма записи передаваемых параметров показана в первом вызове процедуры `add_dept`, приведенном ниже определения процедуры. Первый фактический параметр предоставляет значение `'TRAINING'` для параметра `name`, а второй фактический параметр, равный `2500`, присваивается на основе его позиции формальному параметру `loc`.

Форма записи, в которой используются имена передаваемых параметров, показана в последнем примере. В вызове процедуры указаны ссылки на имена параметров. Параметр `loc`, объявленный вторым формальным параметром, получает фактическое значение, равное `2400`, а параметр `name` –`'EDUCATION'`. Порядок фактических параметров не имеет значения, когда они все заданы.

Примечание. Необходимо предоставить значения всем параметрам, если не заданы значения по умолчанию для формальных параметров. Определение значений по умолчанию будет рассмотрено далее.

Использовании опции DEFAULT в определении параметров

- **Задание значений параметров по умолчанию:**

```
CREATE OR REPLACE PROCEDURE add_dept(
 name departments.department_name%TYPE:='Unknown',
 loc  departments.location_id%TYPE DEFAULT 1700)
IS
BEGIN
  INSERT INTO departments (...)
  VALUES (departments_seq.NEXTVAL, name, loc);
END add_dept;
```

- **Возможность гибкого комбинирования методов передачи параметров (позиционного и по имени):**

```
EXECUTE add_dept
EXECUTE add_dept ('ADVERTISING', loc => 1200)
EXECUTE add_dept (loc => 1200)
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Использовании опции DEFAULT в определении параметров

В коде, приведенном на слайде, показаны два способа задания значения по умолчанию параметру типа IN. В этих способах используются:

- оператор присваивания (:=), как это показано для параметра name;
- опция DEFAULT, как это показано для параметра loc.

Если для формальных параметров заданы значения по умолчанию, процедуру можно вызвать без фактических параметров. В таком случае подпрограмме можно передать различное количество фактических параметров, оставляя или переопределяя значения по умолчанию. Рекомендуется вначале объявить параметры без значений по умолчанию. Затем можно добавить формальные параметры, имеющие значения по умолчанию,. которые не надо изменять при каждом вызове процедуры.

Примечание: нельзя задать значения по умолчанию для параметров типа OUT и IN OUT.

На слайде показаны три способа вызова процедуры:

- В первом примере каждому параметру присваивается значение по умолчанию;
- Во втором примере демонстрируется комбинированная запись позиционных и именованных параметров. Использование формы записи, в которой значение параметру loc присваивается по его имени, представлено в качестве примера.
- В последнем примере для параметра name используется значение по умолчанию, а для параметра loc предоставляется значение.

Использовании опции `DEFAULT` для параметров (продолжение)

Обычно форма записи с именами параметров используется для переопределения значений по умолчанию формальных параметров. Однако фактический параметр должен указываться, когда для формального параметра нет значения по умолчанию.

Примечание: в вызове подпрограммы все позиционные параметры должны предшествовать параметрам, задаваемым по имени, иначе будет получено сообщение об ошибке, как это показано в следующем примере:

```
EXECUTE add_dept(name=>'new dept', 'new location')
```

В результате выдаются такие сообщения об ошибках :

```
ERROR at line 1:
ORA-06550: line 1, column 7:
PLS-00306: wrong number or types of arguments in call to
'ADD_DEPT'
ORA-06550: line 1, column 7:
PL/SQL: Statement ignored
```

Обобщение режимов передачи параметров

IN	OUT	IN OUT
Режим по умолчанию	Должен быть задан	Должен быть задан
Значение передается в подпрограмму	Возвращается в вызывающую среду	Передается в подпрограмму; возвращается в вызывающую среду
Формальный параметр выполняет роль константы	Неинициализированная переменная	Инициализированная переменная
Фактический параметр может быть литералом, выражением, константой или инициализированной переменной	Должен быть переменной	Должен быть переменной
Может быть присвоено значение по умолчанию	Не может быть присвоено значение по умолчанию	Не может быть присвоено значение по умолчанию

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обобщение режимов передачи параметров

По умолчанию для параметра действует режим `IN`. Когда режим явно не указывается, его значение – `IN`. Другие режимы должны быть явно указаны (`OUT` или `IN OUT`) в определении каждого из таких параметров.

Формальному параметру `IN` нельзя присвоить значение и в теле процедуры формальный параметр такого типа не может быть изменен. По умолчанию для параметра `IN` передается ссылка. В объявлении формального параметра `IN` может быть задано значение по умолчанию. В этом случае при вызове подпрограммы можно не указывать фактическое значение такого параметра, так как будет действовать значение по умолчанию.

Параметрам `OUT` и `IN OUT` должно быть присвоено значение перед возвратом обратно в вызывающую среду. Для параметров `OUT` и `IN OUT` нельзя задать значения по умолчанию. Чтобы повысить производительность при использовании параметров `OUT` и `IN OUT` можно указать для компилятора подсказку `NOCOPY`, что приведет к передаче параметров по ссылке.

Примечание: использование подсказки `NOCOPY` рассматривается далее в этом курсе.

Вызов процедур

- **Использование анонимного блока**
- **Использование другой процедуры, как в следующем примере:**

```
CREATE OR REPLACE PROCEDURE process_employees
IS
   CURSOR emp_cursor IS
    SELECT employee_id
    FROM   employees;
BEGIN
   FOR emp_rec IN emp_cursor
   LOOP
     raise_salary(emp_rec.employee_id, 10);
   END LOOP;
   COMMIT;
END process_employees;
/
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Вызов процедур

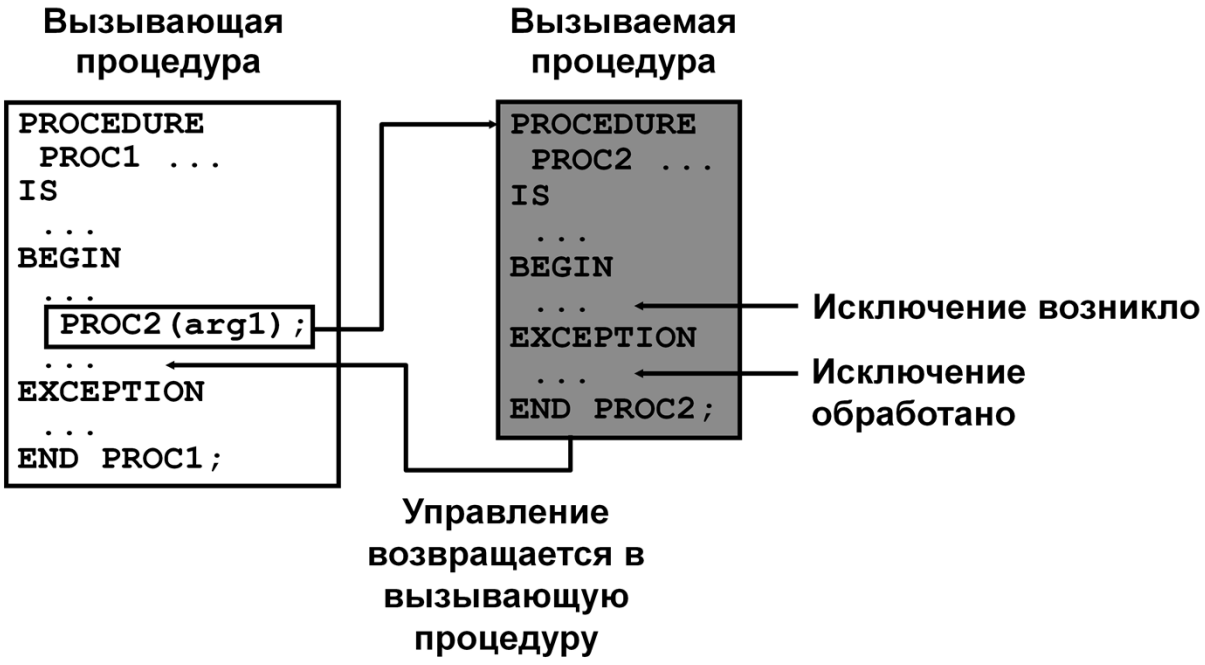
Можно вызвать процедуры, используя:

- анонимные блоки;
- другую процедуру или подпрограмму PL/SQL.

Примеры, которые приводились на предыдущих страницах, иллюстрировали использование анонимных блоков (или команды `EXECUTE` в *i*SQL*Plus).

В примере на слайде показано, как вызвать процедуру из другой хранимой процедуры. Для обработки всех строк таблицы `EMPLOYEES` в хранимой процедуре `PROCESS_EMPS` используется курсор. Каждый выбираемый с помощью этого курсора номер служащего передается процедуре `RAISE_SALARY`, в результате выполнения которой оклад служащего с таким номером повышается на 10%.

Обработанные исключения



Copyright © 2004, Oracle. All rights reserved.

Обработанные исключения

При разработке процедур, вызываемых из других процедур, следует знать о воздействиях, которые обработанные и необработанные исключения оказывают на транзакцию и вызывающую процедуру.

Когда в вызванной процедуре возникает *исключение* (*exception*), управление немедленно передается в секцию обработки исключений текущего блока. Исключение рассматривается как обработанное, когда в секции исключений присутствует обработчик данного исключения.

При возникновении обрабатываемого исключения происходит следующая последовательность действий:

1. Исключение произошло.
2. Управление передается обработчику исключений.
3. Выполнение блока прекращается.
4. Продолжается выполнение вызывающей программы/блока, как будто ничего не произошло.

Любые команды *языка манипулирования данными* (*data manipulation language* – DML), которые были выполнены в вызывающей и вызываемой процедуре, в которой возникло исключение, остаются частью транзакции. Команды DML откатываются, если это откат производится внутри процедуры до исключения.

Примечание: в секции исключений можно явно завершить транзакцию по команде `COMMIT` или `ROLLBACK`.

Пример обработанных исключений

```
CREATE PROCEDURE add_department(
    name VARCHAR2, mgr NUMBER, loc NUMBER) IS
BEGIN
  INSERT INTO DEPARTMENTS (department_id,
    department_name, manager_id, location_id)
  VALUES (DEPARTMENTS_SEQ.NEXTVAL, name, mgr, loc);
  DBMS_OUTPUT.PUT_LINE('Added Dept: '||name);
EXCEPTION
 WHEN OTHERS THEN
  DBMS_OUTPUT.PUT_LINE('Err: adding dept: '||name);
END;
```

```
CREATE PROCEDURE create_departments IS
BEGIN
  add_department('Media', 100, 1800);
  add_department('Editing', 99, 1800);
  add_department('Advertising', 101, 1800);
END;
```

Copyright © 2004, Oracle. All rights reserved.

Пример обработанных исключений

Две приведенные на слайде процедуры выполняют следующее:

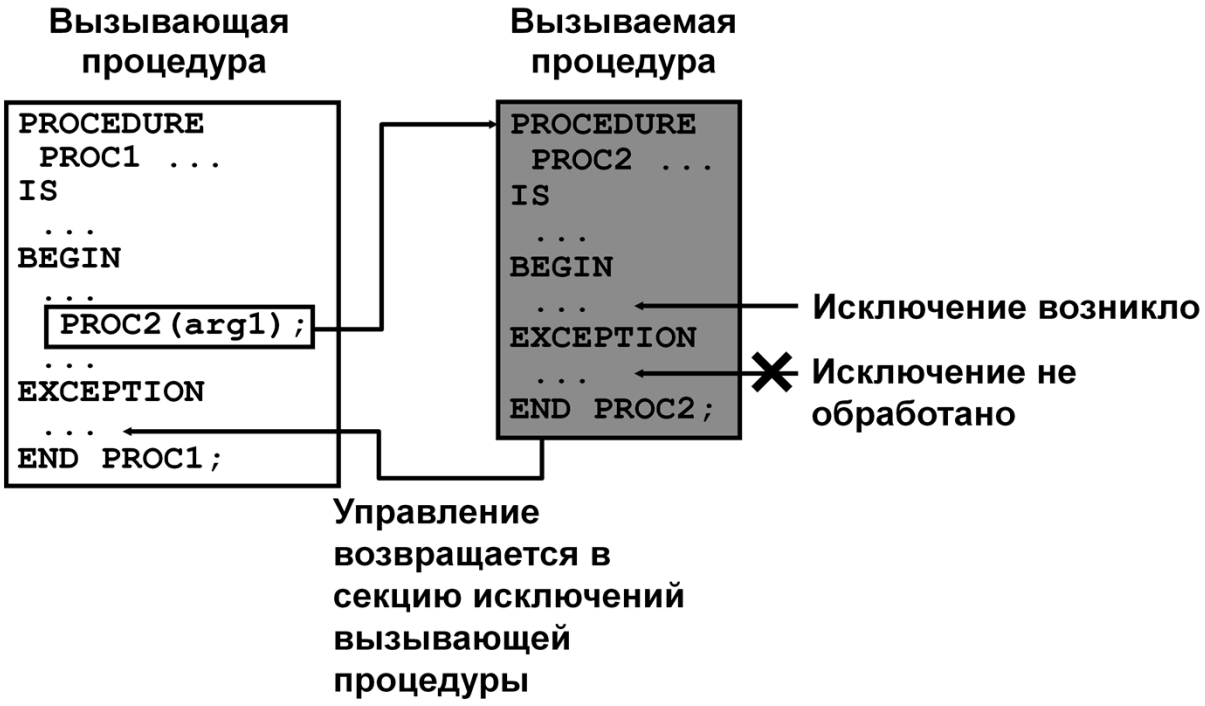
- Процедура `add_department` создает запись о новом отделе, получая для этого из последовательности номер нового отдела и используя параметры `name`, `mgr` и `loc` для задания значений столбцов `department_name`, `manager_id` и `location_id`.
- Процедура `create_departments` вставляет записи о нескольких отделах, используя процедуру `add_department`.

Процедура `add_department` перехватывает все возникающие исключения в собственном обработчике. При выполнении процедуры `create_departments` выдаются следующие результаты:

```
Added Dept: Media
Err: Adding Dept: Editing
Added Dept: Advertising
```

Отдел `Editing` с номером менеджера (`manager_id`), равным `99`, не был вставлен, поскольку нарушается ограничение ссылочной целостности для столбца `manager_id`. В связи с тем, что исключение было обработано в процедуре `add_department`, процедура `create_department` продолжает выполнение. Запрос к таблице `DEPARTMENTS` при условии `location_id = 1800` показывает, что были добавлены отделы `Media` и `Advertising` (но нет строки об `Editing`).

Необработанные исключения



Copyright © 2004, Oracle. All rights reserved.

Необработанные исключения

Как было указано раньше, при возникновении исключения в вызываемой процедуре управление немедленно передается в секцию исключений текущего блока. Если нет соответствующего обработчика, исключение не обрабатывается. Выполняется следующая последовательность действий:

1. Исключение произошло.
2. Выполнение блока прекращается, поскольку отсутствует обработчик исключения; результаты операций DML, произведенных внутри процедуры, не откатываются.
3. Исключение распространяется в секцию исключений вызывающей процедуры. В результате управление возвращается в секций исключений вызывающего блока, если она существует.

Если исключение не обрабатывается, тогда для всех команд DML вызывающей и вызываемой процедур производится откат изменений, который затрагивает и все изменения, сделанные в хост-переменных.

Исключение не оказывает влияние на команды DML, которые были выполнены перед вызовом кода PL/SQL, в котором произошло и не было обработано возникшее исключение.

Пример необработанных исключений

```
CREATE PROCEDURE add_department_noex(
    name VARCHAR2, mgr NUMBER, loc NUMBER) IS
BEGIN
  INSERT INTO DEPARTMENTS (department_id,
    department_name, manager_id, location_id)
  VALUES (DEPARTMENTS_SEQ.NEXTVAL, name, mgr, loc);
  DBMS_OUTPUT.PUT_LINE('Added Dept: '||name);
END;
```

```
CREATE PROCEDURE create_departments_noex IS
BEGIN
  add_department_noex('Media', 100, 1800);
  add_department_noex('Editing', 99, 1800);
  add_department_noex('Advertising', 101, 1800);
END;
```

Copyright © 2004, Oracle. All rights reserved.

Пример необработанных исключений

В примере на слайде показана процедура `add_department_noex`, не содержащая секцию исключений. В данном случае исключение возникает, когда добавляется отдел `Editing`. Вследствие отсутствия обработчиков исключений в обеих подпрограммах никакие записи о новых отделах не добавляются в таблицу `DEPARTMENTS` table. При выполнении процедуры `create_departments_noex` выдаются результаты, подобные приведенным ниже:

```
Added Dept: Media
BEGIN create_departments_noex; END;

*
ERROR at line 1:
ORA-02291: integrity constraint (ORA1.DEPT_MGR_FK) violated
- parent key not
found
ORA-06512: at "ORA1.ADD_DEPARTMENT_NOEX", line 4
ORA-06512: at "ORA1.CREATE_DEPARTMENTS_NOEX", line 4
ORA-06512: at line 1
```

Хотя выводимые данные показывают, что была добавлена запись об отделе `Media`, результат этой операции откатывается, поскольку исключение не было обработано ни в одной из вызванных подпрограмм.

Удаление процедур

Удаление процедуры, хранимой в базе данных.

- **Синтаксис:**

```
DROP PROCEDURE имя_процедуры
```

- **Пример:**

```
DROP PROCEDURE raise_salary;
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Удаление процедур

Если хранимая процедура больше не нужна, ее можно удалить по команде SQL `DROP PROCEDURE`.

Примечание: независимо от того, удачно или неудачно завершается команда *языка определения данных* (*data definition language*, DDL), например `DROP PROCEDURE`, для любой отложенный транзакции выполняется фиксация (commit) и откат невозможен.

Просмотр информации о процедурах в словаре данных

Сведения о процедурах PL/SQL хранятся в следующих представлениях словаря данных:

- **USER_SOURCE содержит исходный код подпрограмм текущего пользователя; ALL_SOURCE дополнительно хранит подпрограммы других пользователей, предоставивших текущему пользователю привилегию их выполнения (EXECUTE).**

```
SELECT text
FROM   user_source
WHERE  name='ADD_DEPARTMENT' and type='PROCEDURE'
ORDER BY line;
```

- **USER_OBJECTS содержит имена процедур, принадлежащих текущему пользователю.**

```
SELECT object_name
FROM   user_objects
WHERE  object_type = 'PROCEDURE';
```

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Просмотр информации о процедурах в словаре данных

В таблицах словаря данных хранится исходный код подпрограмм PL/SQL, скомпилированных успешно или с ошибками. Для просмотра исходного кода используйте запросы (SELECT) к следующим представлениям:

- USER_SOURCE (для вывода кода PL/SQL, принадлежащего текущему пользователю);
- ALL_SOURCE (для вывода кода PL/SQL, принадлежащего другим пользователям, предоставившим текущему пользователю привилегию EXECUTE для выполнения своей подпрограммы).

В примере на слайде указаны следующие столбцы представления USER_SOURCE:

- TEXT (содержит строку исходного кода PL/SQL);.
- NAME (наименование подпрограммы в символах верхнего регистра);
- TYPE (тип подпрограммы; например, PROCEDURE или FUNCTION);.
- LINE (номер для каждой строки исходного кода).

Кроме этих столбцов в представлении ALL_SOURCE содержится столбец OWNER (владелец).

Примечание. Нельзя вывести код встроенных PL/SQL-пакетов, а также исходный код PL/SQL, обработанный утилитой WRAP, которая конвертирует исходный код PL/SQL в зашифрованный вид.

Преимущества подпрограмм

- Упрощается сопровождение
- Повышается безопасность и целостность данных
- Повышается производительность
- Код становится более ясным

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Преимущества подпрограмм

Модульная разработка кода приложений с использованием процедур и функций имеет целый ряд преимуществ.

- **Упрощение сопровождения** (**easy maintenance)** обеспечивается благодаря тому, что подпрограммы размещены в одном месте. Модифицирование одной подпрограммы оказывает влияние на несколько приложений и позволяет избежать излишних затрат при тестировании.
- **Повышение безопасности данных (improved data security)** может быть достигнуто путем контроля косвенного доступа непривилегированных пользователей к объектам базы данных без явного предоставления каких-либо привилегий. Подпрограммы выполняются по умолчанию с правами создателя. Привилегия выполнения не предоставляет пользователю, вызывающему подпрограмму, прямой доступ к объектам, доступным подпрограмме.
- **Целостность данных (data integrity)** обеспечивается за счет того, что связанные действия либо выполняются все вместе, либо не выполняются вообще.
- **Повышение производительности (improved performance)** может быть получено вследствие повторной используемости разобранного кода PL/SQL, располагаемого в разделяемой области SQL на сервере. Последующий вызов подпрограммы не повлечет снова разбор кода. Поскольку код PL/SQL разбирается во время компиляции, устраняются накладные расходы, вызванные разбором команд SQL, во время выполнения. Благодаря объединению SQL-команд в подпрограммы уменьшается количество вызовов базы данных и сетевой трафик.
- **Улучшение ясности кода (improved code clarity);** использование соответствующих имен идентификаторов и правил для описания действий подпрограммы снижает потребность в комментировании кода и повышает его ясность.

Итоги

- Создание процедуры для выполнения задачи или действия
- Создание, компиляция и сохранение процедур в базе данных с использованием команды SQL `CREATE PROCEDURE`
- Передача данных из вызывающей среды в процедуру с помощью параметров в трех различных режимах: `IN` (по умолчанию), `OUT` и `IN OUT`
- Влияние обработанных и необработанных исключений на транзакции и вызывающие процедуры

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги

Процедура – это подпрограмма, выполняющая определенные действия. Вы можете скомпилировать и сохранить процедуру в базе данных. Процедура может совсем не возвращать или вернуть значения в вызывающую среду. Существует три типа (режима) параметров: `IN`, `OUT` и `IN OUT`.

Следует знать о влиянии обработанных и необработанных исключений на транзакцию и вызывающую процедуру. Исключение обрабатывается в секции исключений подпрограммы.

Итоги

- **Удаление процедур из базы данных с помощью команды SQL `DROP PROCEDURE` .**
- **Создание модульного кода приложения с помощью процедур, используемых в качестве строительных блоков.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Итоги (продолжение)

Процедуры можно модифицировать и удалять.

Процедуры – это модульные компоненты, формирующие строительные блоки приложения.

Процедуры можно также создавать на стороне клиента и использовать в клиентских приложениях.

Обзор практического занятия 1

- **Создание хранимых процедур для выполнения следующих действий:**
  - **вставка новых строк в таблицу с использованием передаваемых значений параметров;**
  - **обновление, удаление и выборка строк таблицы, удовлетворяющих передаваемым значениям параметров.**
- **Обработка исключений в процедуре.**
- **Компиляция и вызов процедур.**

ORACLE

Copyright © 2004, Oracle. All rights reserved.

Обзор практического занятия 1

При выполнении заданий этого практического занятия Вы будете создавать процедуры, содержащие команды DML и запросы.

Используйте команду *i*SQL*Plus `SHOW ERRORS` для получения информации об ошибках при их возникновении в ходе выполнения практики.

Исправляйте ошибки, сообщения о которых получены в *i*SQL*Plus, в первоначальном скрипт-файле, а не в буфере. Затем повторно выполняйте исправленную версию файла, в результате чего новый код процедуры сохраняется в словаре данных.

Practice 1

Примечание. Описания таблиц и примеры данных могут быть найдены в приложении B “Структура таблиц и данные.” Сохраняйте подпрограммы в локальной файловой системе в файлах с расширением `.sql`, используя кнопку `Save Script`.

Не забудьте установить для параметра `SERVEROUTPUT` значение `ON`.

1. Создайте и выполните процедуру `ADD_JOB`. Проверьте результаты.
   (а). Создайте процедуру с именем `ADD_JOB` для вставки новой должности в таблицу `JOBS`. Передайте процедуре с помощью двух параметров идентификатор (`JOB_ID`) и наименование должности..
   (б). Откомпилируйте код и вызовите процедуру, передав ей следующие данные о должности: идентификатор `IT_DBA` и наименование `Database Administrator`. Просмотрите результаты с помощью запроса к таблице `JOBS` .

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_DBA	Database Administrator		

   (в). Вызовите процедуру еще раз, передав ей значения `ST_MAN` и `Stock Manager`. Что произошло и почему?

   _____

   _____

2. Создайте процедуру `UPD_JOB` для изменения должности в таблице `JOBS` .
   (а). Создайте процедуру `UPD_JOB` для изменения наименования должности. Передайте процедуре с помощью двух параметров идентификатор (`JOBID`) и новое наименование должности. Возбудите исключение, если не выполняется изменение.
   (б). Compile Откомпилируйте код. Вызовите процедуру, чтобы измененить на `Data Administrator` наименования должности с идентификатором `IT_DBA`. Просмотрите результаты с помощью запроса к таблице `JOBS`.

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
IT_DBA	Data Administrator		

   идентификатором `IT_WEB` и наименованием `Web Master`).

3. Создайте процедуру `DEL_JOB` для удаления должности из таблицы `JOBS`.
   (а). Создайте процедуру `DEL_JOB` для удаления должности. Возбудите исключение, если нет удаляемой должности.

   .

Практическое занятие 1 (продолжение)

(б). Откомпилируйте код; вызовите процедуру, передав ей идентификатор должности `IT_DBA`. Выполните запрос к таблице `JOBS`, чтобы просмотреть результаты.

no rows selected

Кроме того, проверьте возникновение исключения, выполнив удаление несуществующей должности (используйте должность с идентификатором `IT_WEB`). Вы должны получить в выходных данных сообщение, генерируемое при возбуждении исключения.

4. Создайте процедуру `GET_EMPLOYEE`, в которой производится запрос к таблице `EMPLOYEES` и выбирается оклад и идентификатор должности служащего с заданным номером.

   (а). Создайте процедуру, которая возвращает значение столбцов `SALARY` и `JOB_ID` для служащего с заданным номером. Откомпилируйте код и исправьте синтаксические ошибки.

   (б). Выполните процедуру, используя хост-переменные в качестве двух параметров в режиме `OUT`: один для оклада и второй для идентификатора должности. Выведите оклад и идентификатор должности для служащего с номером 120.

SALARY
8000

(в).

JOB
ST_MAN

то

_____

_____